Database Management Systems - Mini Project Report

BloodLink: Blood Bank Management System

Submitted by: _____ (Name) Roll No: _____ Division: ____
Course: Database Management Systems
Faculty: Dr. Mrudula Gudadhe
GitHub: https://github.com/<your-username>/bloodlink

1. Introduction

Blood banks must know, at every moment, how many units of each blood group are available, which donors are eligible to donate, and whether a hospital request can be met. Errors in this data can cost lives. **BloodLink** is a database-driven web application that records donors, donations, hospitals and blood requests, and keeps the blood stock consistent automatically using triggers. The SQLite database runs inside the browser (sql.js), so all SQL queries can be executed live without any server.

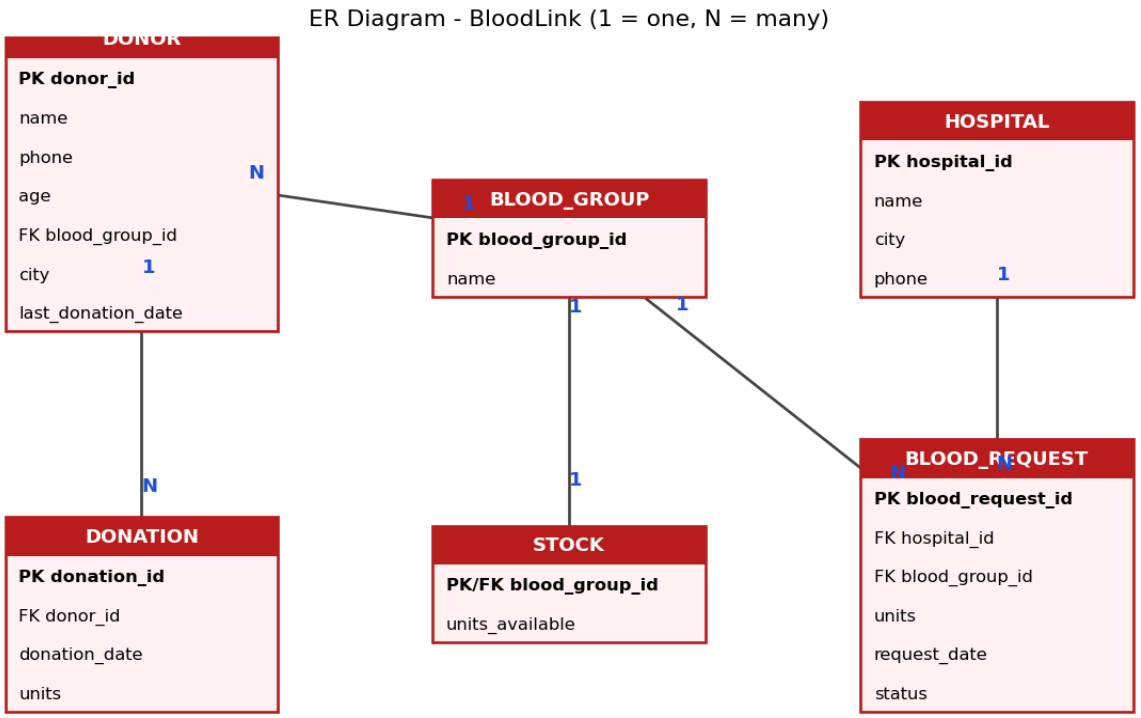
2. Problem Statement

To design a relational database for a blood bank that (a) stores donor, hospital, donation and request data, (b) automatically updates stock when blood is donated or issued, (c) prevents a donor from donating again within 90 days, and (d) prevents issuing blood when stock is insufficient.

3. Objectives

- Design a normalized (3NF) schema with proper keys and integrity constraints.
- Provide CRUD operations for all entities through a web interface.
- Enforce business rules with triggers (donor eligibility, stock update, stock check).
- Use views, joins, subqueries and aggregate queries for reports.
- Demonstrate transactions with BEGIN / COMMIT / ROLLBACK.
- Provide a live in-browser SQL console.

4. ER Diagram



Relationships: a Blood Group has many Donors and has exactly one Stock row (1:1); a Donor makes many Donations; a Hospital raises many Blood Requests; a Blood Request is for one Blood

Group. All other relationships are 1:N.

5. Database Schema

Table	Columns	Constraints
blood_group	blood_group_id, name	PK; name UNIQUE and CHECK (A+, A-, B+, B-, O+, O-, AB+, AB-)
stock	blood_group_id, units_available	PK and FK to blood_group (CASCADE); units_available >= 0
donor	donor_id, name, phone, age, blood_group_id, city, last_donation_date	PK; phone UNIQUE; age CHECK 18-65; FK to blood_group
hospital	hospital_id, name, city, phone	PK; phone UNIQUE
donation	donation_id, donor_id, donation_date, units	PK; FK to donor; units CHECK 1-2
blood_request	blood_request_id, hospital_id, blood_group_id, units, request_date, status	PK; FKs to hospital, blood_group; units > 0; status CHECK (Pending/Fulfilled/Rejected)

Normalization: 1NF - atomic values and a primary key in every table. **2NF** - every table has a single-column key, so no partial dependencies. **3NF** - no transitive dependency: blood-group names are stored once in *blood_group* and referenced by key; hospital details are not repeated in requests; stock levels are kept in a separate table keyed by blood group, instead of being repeated in donor or donation rows.

6. Technologies Used

- **Database:** SQLite, executed in the browser through sql.js
- **Front end:** HTML, CSS, JavaScript
- **Version control / hosting:** Git, GitHub, GitHub Pages
- **Tools:** VS Code, matplotlib (ER diagram)

7. SQL Queries and Features Implemented

7.1 Triggers (SQLite has no stored procedures, so business logic is implemented as triggers):

```
-- Rule 1: 90-day gap between donations
CREATE TRIGGER trg_donor_eligibility BEFORE INSERT ON donation
WHEN (SELECT last_donation_date FROM donor WHERE donor_id=NEW.donor_id) IS NOT NULL
 AND julianday(NEW.donation_date) - julianday((SELECT last_donation_date FROM donor
     WHERE donor_id=NEW.donor_id)) < 90
BEGIN SELECT RAISE(ABORT,'Donor not eligible: 90-day gap required'); END;

-- Rule 2: donation increases stock and updates donor's last donation date
CREATE TRIGGER trg_donation_stock AFTER INSERT ON donation
BEGIN
  UPDATE stock SET units_available = units_available + NEW.units
   WHERE blood_group_id=(SELECT blood_group_id FROM donor WHERE donor_id=NEW.donor_id);
  UPDATE donor SET last_donation_date=NEW.donation_date WHERE donor_id=NEW.donor_id;
END;

-- Rule 3: block fulfilling a request when stock is low; Rule 4: reduce stock on issue
CREATE TRIGGER trg_check_stock BEFORE UPDATE OF status ON blood_request
WHEN OLD.status='Pending' AND NEW.status='Fulfilled' AND
 (SELECT units_available FROM stock WHERE blood_group_id=NEW.blood_group_id) < NEW.units
BEGIN SELECT RAISE(ABORT,'Insufficient stock to fulfil this request'); END;

CREATE TRIGGER trg_issue_stock AFTER UPDATE OF status ON blood_request
WHEN OLD.status='Pending' AND NEW.status='Fulfilled'
BEGIN UPDATE stock SET units_available=units_available-NEW.units
      WHERE blood_group_id=NEW.blood_group_id; END;
```

7.2 Views: v_stock, v_donor_details and v_request_details (multi-table joins).

7.3 Sample queries

```
-- Aggregate: donations per blood group
SELECT bg.name, COUNT(*) AS donations, SUM(d.units) AS units
FROM donation d JOIN donor dn USING(donor_id) JOIN blood_group bg USING(blood_group_id)
GROUP BY bg.blood_group_id;

-- Subquery: donors who never donated
SELECT name, phone FROM donor WHERE donor_id NOT IN (SELECT donor_id FROM donation);

-- Low stock alert
SELECT * FROM v_stock WHERE units_available < 2;
```

7.4 Transaction: recording a donation and issuing blood to a hospital can be done atomically. If a statement fails (for example a trigger aborts), the console issues ROLLBACK and no partial change is saved.

```
BEGIN;
INSERT INTO donation(donor_id,donation_date,units) VALUES (9,'2026-10-11',1);
UPDATE blood_request SET status='Fulfilled' WHERE blood_request_id=2;
COMMIT;
```

7.5 Other features: PK/FK constraints with RESTRICT and CASCADE, CHECK and UNIQUE constraints, CRUD for all tables, dashboard, live SQL console with 10 sample queries, database reset.

8. Screenshots

[Paste screenshot here: Dashboard with blood stock]

[Paste screenshot here: Donors CRUD page]

[Paste screenshot here: Blood request page]

[Paste screenshot here: SQL Console running a JOIN query]

[Paste screenshot here: Trigger error: donor not eligible / insufficient stock]

9. Conclusion

BloodLink demonstrates ER modelling, normalization, integrity constraints, SQL joins, subqueries and aggregation, views, triggers and transactions. Triggers keep the blood stock accurate and enforce medical rules directly inside the database, so the rules hold no matter how data is entered.

10. Future Scope

- Migrate to MySQL/PostgreSQL with stored procedures and a Node.js / Flask back end.
- Blood unit expiry tracking and automatic expiry alerts.
- SMS / email notification to eligible donors when stock is low.
- Role-based login for admin, donors and hospitals.